Marcura

Full Stack Developer — Technical Assessment

Exchange Rate Management System

Role	Senior Full Stack Engineer
Submission	GitHub repository + short screen recording (3–5 min)
Stack	Java 17+ · Spring Boot · Angular · Spring AI · AI Coding Tools

1. Overview

This assessment asks you to design and build an Exchange Rate Management System end-to-end — backend API, scheduled data collection, Angular frontend, and a small AI-powered insight feature. It is intentionally open-ended. There is no single correct architecture. What we are evaluating is how you think, how you work, and how effectively you use AI tools as a genuine part of your development process.

Read the full brief before writing any code. The grading rubric in Section 8 tells you explicitly where the marks are concentrated.

2. Problem Statement

Marcura needs an internal platform to consolidate how exchange rates are calculated and presented across its applications. The platform must collect live rates, apply a spread-adjusted calculation, serve a REST API, surface the data through an Angular dashboard, and provide a lightweight AI-generated insight layer on top of historical rate trends.

The source of exchange rates is https://fixer.io/ (free subscription). Rates should be fetched once a day and stored locally so the application does not depend on the external API for every query.

3. Technology Requirements

The following choices are fixed. Everything else — project structure, libraries, patterns — is your decision.

- Backend: Java 17 or later, Spring Boot, Maven, Hibernate / Spring Data JPA, any relational database
- Frontend: Angular v15 or later, TypeScript throughout
- AI Integration: Spring AI (preferred) or LangChain4j, connected to any open-source LLM (Ollama, a local model, or an OpenAI-compatible endpoint)

- AI Coding Tools: at least one AI coding assistant must be demonstrably part of your workflow — Claude Code, Cursor, GitHub Copilot, or equivalent
- API Documentation: Swagger / OpenAPI

4. Backend Requirements

The sections below describe the behaviour expected from the backend. Project structure, layering, libraries, and patterns are your decisions. Backend code quality — separation of concerns, idiomatic use of Spring and JPA, appropriate use of types and numeric precision, and adherence to engineering standards — is graded under the Backend rubric items in Section 10.

4.1 Data Collection

Implement a scheduled job that fetches the latest exchange rates from Fixer.io once a day at 12:05 AM GMT and persists them to the database. The stored record must include the currency code, the rate value, and the date the rate was calculated as reported by the API — not the system date of the fetch. Duplicate rates for the same currency and date must be handled gracefully. Assume the service may run as multiple instances in production; the scheduler should behave correctly in that environment, and the approach and its justification matter more than the specific mechanism.

4.2 Exchange Rate API

Expose a REST endpoint that accepts a source currency, a target currency, and an optional date, and returns the spread-adjusted exchange rate for that pair. The calculation must use the rate data held locally in the database. When no date is supplied, use the most recent available rates. When rates for a requested date do not exist, return an appropriate HTTP error.

Every successful query must increment a usage counter for each of the two currencies involved. This counter is used by the analytics features. The increment must be safe under concurrent requests.

The spread-adjusted calculation is described in Section 6.

4.3 Analytics Endpoint

Expose an endpoint that returns usage statistics — at minimum, the query count per currency and the dates on which queries were made. The structure of this response is your design decision; it needs to support the frontend analytics view described in Section 5.

4.4 Manual Refresh (Optional)

An additional endpoint to manually trigger a rate fetch and upsert — without affecting existing usage counters — is an optional extension.

5. Frontend Requirements

The Angular application should be a clean, navigable single-page application that consumes your backend API. Three views are required.

5.1 Exchange Rate Calculator

A view where a user can select two currencies, optionally specify a date, and see the spread-adjusted exchange rate returned by your API. The form should behave sensibly — validated inputs, clear error messages when the API returns an error, and a visible loading state while the request is in flight.

5.2 Historical Rates & Trend Chart

A view that allows the user to select a currency pair and a date range and see two things side by side: a table of the raw exchange rates for that period, and a line chart showing how the rate moved over time. The chart does not need to be elaborate — clarity of the trend is what matters. Below or alongside the chart, display the AI-generated trend insight described in Section 7.

5.3 Usage Analytics Dashboard

A view that surfaces the data from your analytics endpoint — which currencies are queried most often, over what time periods, and any patterns visible in the usage data. How you choose to visualise this is up to you.

5.4 Frontend Standards

The application must run with ng serve pointed at your backend via a configurable environment variable, so reviewers can run it locally without code changes. Frontend code quality — component design, separation of concerns, type safety, and adherence to Angular best practices — is graded under the Frontend rubric items in Section 10. How you structure the application is your call.

6. Exchange Rate Calculation

6.1 Formula

Apply the spread of whichever currency in the pair carries the higher spread:

Spread-Adjusted Rate
`adjustedRate = (toCurrencyRateToUSD / fromCurrencyRateToUSD) × ((100 − MAX(toSpread, fromSpread)) / 100)`

6.2 Worked Example

EUR	PLN

Rate to USD	0.8	3.7
Spread	1%	4%

```
(3.7 / 0.8)  ×  ((100 − 4) / 100)  =  4.625  ×  0.96  =  4.44
```

7. AI-Powered Trend Insight

7.1 What to Build

When a user views the Historical Rates & Trend Chart (Section 5.2), a short natural language insight about the rate trend for the selected period should be generated and displayed alongside the chart. This insight should be produced by calling an LLM through Spring AI (or LangChain4j), with the historical rate data for that period passed as context.

The insight should be a brief, readable commentary — something a user glancing at the chart would find useful. It does not need to be financially precise or sophisticated. What matters is that the LLM is genuinely receiving the rate data and responding to it, not producing a generic response.

7.2 Technical Expectations

Use Spring AI's chat client abstraction or an equivalent to call a locally-running open-source model (Ollama is the simplest setup) or any OpenAI-compatible endpoint. The model choice is yours. The key requirements are:

- The rate data for the selected period must be injected into the prompt as context — the LLM must be reading the actual numbers, not guessing.
- The system prompt must be designed to constrain the output to a concise, relevant insight — prompt design quality will be assessed.
- The backend must expose an endpoint that the Angular frontend calls to retrieve this insight. The frontend should display it cleanly, with an appropriate loading state.
- Document your model setup in the README so reviewers can run it locally without configuration guesswork.

7.3 What We Are Not Expecting

We are not expecting financial accuracy, a fine-tuned model, or a production-grade RAG pipeline. We are looking for correct Spring AI wiring, a thoughtful prompt, and a feature that actually works end-to-end. A well-integrated simple solution scores higher than an over-engineered one that does not run.

8. AI-Augmented Development

Using AI coding tools is a mandatory expectation for this role. However, what we are evaluating is not whether you used AI — it is the quality of how you used it. The distinction we draw is between using AI as a smarter autocomplete versus using it as a genuine workflow automation layer.

8.1 What Good Looks Like

We expect AI tools to have been involved across the full development cycle, not just for generating boilerplate. This means: using an agent to help plan and break down the work before implementation starts, running agentic sessions that produce coherent multi-file outputs rather than isolated snippets, using AI to generate your test coverage rather than writing tests manually, and using AI to produce documentation you then reviewed and corrected.

The most useful signal for reviewers is not how much code the AI wrote — it is whether you configured it thoughtfully (context files, custom commands, workspace setup), directed it effectively, and applied critical judgment to its output.

8.2 Required Evidence

Your repository must contain:

1. A PLAN.md or equivalent planning artefact produced with AI assistance before implementation started.
2. Any AI tool configuration files committed — CLAUDE.md, .cursor/rules, Copilot workspace config, custom slash commands, or equivalent. An empty or missing configuration is a signal that AI was used superficially.
3. A README section titled "AI Workflow" describing which tool you used, how you configured it, and at least one example where the agent produced something you disagreed with and what you did about it.
4. Commit history that makes AI-assisted phases visible — use a consistent prefix such as [AI] so reviewers can trace the contribution without reading every diff.

8.3 What Will Be Assessed

Reviewers will look at the repository as a window into your daily working style. They will ask: does this candidate treat AI as a workflow tool or a crutch? Is there evidence of genuine agent use — multi-step, context-aware, iterated — or just single-prompt generation? Did the candidate override the AI at any point, and can they explain why?

9. Submission Requirements

- A GitHub repository (public or private with recruiter access) with a meaningful commit history.
- A README covering: local setup and run instructions, architecture overview, AI Workflow section (Section 8.2), assumptions made, and known trade-offs.
- A 3–5 minute screen recording showing the running application and at least one AI agent session — live or narrated replay. Showing the process matters as much as showing the product.

10. Grading Rubric

This rubric is used by the review panel after submission. Weights reflect the priorities of the role as defined in the job description. The rubric is shared with candidates so they can allocate their time accordingly.

Assessment Area	JD Competency Mapped	Weight	What We Look For
BACKEND	Java & Backend Frameworks	**25%**	Assessed as a group
Core API correctness	Spring MVC · REST design	8%	Endpoint behaviour, HTTP semantics, correct formula, date handling, 404 on missing rates
Data persistence & scheduler	JPA · Hibernate · Spring Scheduler	6%	Upsert correctness, date field from API response, scheduler correctness under multi-instance deployment
Concurrency & thread safety	JVM concurrency awareness	5%	Counter increment safe under concurrent requests — approach and justification matter more than the specific solution
Code quality	Engineering standards	6%	BigDecimal usage, separation of concerns, named queries, no obvious code smells
FRONTEND	Angular · TypeScript	**20%**	Assessed as a group
Calculator view	Angular forms · HTTP client	6%	Form validation, error handling, loading states, typed models
Historical rates & trend chart	Data visualisation · Angular	8%	Chart clarity, table implementation, integration with AI insight panel
Analytics dashboard	Component design	6%	Meaningful visualisation of usage data, clean component architecture
AI TREND INSIGHT	Spring AI · LLM integration	**20%**	Assessed as a group
Spring AI wiring	Spring AI · LangChain4j	8%	Correct use of chat client abstraction, model configured and runnable locally
Prompt design	Prompt engineering	7%	Rate data injected as context, output constrained to relevant insight, not generic filler
Frontend integration	Angular · API design	5%	Insight displayed with loading state, endpoint design sensible
AI-AUGMENTED WORKFLOW	AI-Augmented Development	**25%**	Assessed as a group

Planning artefact	AI as planning tool	5%	PLAN.md or equivalent present, reflects genuine AI-assisted breakdown before implementation
Tool configuration	Agent setup & context design	8%	CLAUDE.md / .cursor/rules / custom commands present and substantive — not placeholder files
Agentic workflow evidence	Autonomous task execution	7%	Commit history and README show multi-step agent use, not just snippet generation
Critical AI use	Judgment & responsible use	5%	Evidence of overriding or correcting AI output, with explanation — passive acceptance scores low
OVERALL ENGINEERING	General engineering	**10%**	Assessed as a group
API documentation	Swagger / OpenAPI	3%	All endpoints documented, Swagger UI functional
Testing	JUnit · Mockito · Angular tests	4%	Coverage of spread calculation logic and at least one integration test — AI-generated test suites are expected
README & documentation quality	Communication	3%	Setup instructions work, architecture is explained, assumptions are stated
TOTAL		**100%**	Pass threshold for interview progression: 60%. Outstanding AI workflow can compensate for incomplete features.

Note for reviewers: the AI Workflow category (25%) is intentionally weighted equally with Backend. A candidate who completes all backend and frontend requirements but shows no substantive AI workflow evidence cannot exceed 75%. A candidate with strong AI workflow practices and a partially complete application may still progress if their process signals are strong.

Appendix A — API Response Formats

GET /exchange

```
{
  "from":           "EUR",
  "to":             "PLN",
  "exchange":       4.4405487565413254,
  "date":           "2024-03-15",
```

```
  "fromQueryCount": 142,
  "toQueryCount":   37
}
```

GET /analytics (suggested shape — your design)

```
{
  "topCurrencies": [
    { "currency": "EUR", "totalCount": 142, "lastQueried": "2024-03-15" },
    { "currency": "USD", "totalCount": 98,  "lastQueried": "2024-03-14" }
  ]
}
```

GET /exchange/insight (suggested shape — your design)

```
{
  "from":    "EUR",
  "to":      "GBP",
  "fromDate": "2024-02-01",
  "toDate":   "2024-03-01",
  "insight": "EUR/GBP softened by approximately 1.8% over this period, with
the
              steepest decline in the final week of February."
}
```

Appendix B — Currency Spread Reference

Currency Group	Spread %
Base Currency (as returned by your Fixer.io API key)	0.00%
JPY, HKD, KRW	3.25%
MYR, INR, MXN	4.50%
RUB, CNY, ZAR	6.00%
All other currencies	2.75%